Day 12/21: Object-Oriented Programming (OOP)

The Data Refinery Sprint | Author: Prasad S Hiremath | Topic: Classes, Objects & Data Pipeline Models

⚡ **Daily Motivation & Post-Dasara Sprint Continuation**
Welcome back from the festive Dasara break! As we enter the second half of our 21-day sprint, we transition from writing isolated procedural functions to building structured, enterprise-grade data architectures. Mastering Object-Oriented Programming unlocks how PySpark DataFrames, Airflow Operators, and BigQuery SDKs are constructed under the hood.

🧱 **Today's Goal: Learn to structure, validate, and encapsulate data stream records into robust Python classes!**

🏢 Real-World Case Study: The Spaghetti Code Pipeline

At **InstaDrop**, engineers processed incoming orders using raw, un-enforced dictionaries and standalone helper scripts. As new channels (Web, Mobile, POS) added fields like tax rates and discount flags, schema drift caused frequent runtime `KeyError` crashes. Senior engineers resolved this by converting unstructured payloads into typed **Domain Model Classes** that self-validate upon instantiation.

💡 Unstructured Dictionary vs. Encapsulated Class Model

❌ **Procedural Dictionary (Fragile)**

```
        # Unstructured dict without schema rules order_1 = {"id": 101, "amt": 250.0} def vali
```

📱 Quick Reference: Paradigm Shift in Data Engineering

Concept	Procedural / Scripting	Object-Oriented Programming (OOP)
Data Unit	Loose dicts, tuples, or primitive variables	Instantiated Class Objects (e.g., `Record`, `Batch`)
Schema Safety	Manual dict key lookup (prone to key error)	Explicit instance attributes initialized in `__init__`
Validation	External multi-nested functions	Encapsulated instance methods (e.g., `record.validate()`)
Pipeline State	Global variables mutated across scripts	Internal object state (e.g., `self.is_valid = True`)

💻 Medium-Level Coding Challenge: InstaDrop Record Validator

> **Task:** Implement a class `DataRecord(record_id, raw_payload)` that validates required fields (`user_id`, `price`) and converts them into typed attributes inside a `clean_data` dictionary.

```
class DataRecord: """Data Pipeline Record Model for InstaDrop Ingestion Engine.""" def __
```

❓ Part 1: Core Practice MCQs

Q1: What is the explicit role of the 'self' parameter in a Python class method?

A) Connects to an external DB handle | B) References the specific object instance calling the method
C) Compiles the class to C-code | D) Declares global pipeline state

Key: B — 'self' explicitly binds methods and attributes to the instance calling the class.

Q2: When is Python's '__init__' constructor method executed?

A) Automatically when a new instance is created | B) Only when explicitly called by user
C) During garbage collection | D) When importing the module

Key: A — Python invokes __init__ automatically upon object instantiation to bind attributes.

❓ Core Practice MCQs (Continued)

Q3: How do Class attributes differ from Instance attributes?

A) Class attributes are unique per instance | B) Class attributes are shared across all instances; instance attributes belong to self
C) Instance attributes cannot be changed | D) They are functionally identical

Key: B — Attributes defined at class level are shared; attributes attached to 'self' belong to individual objects.

Q4: Why do modern data frameworks (PySpark, Airflow) use OOP encapsulation?

A) Eliminates RAM usage completely | B) Binds schema and operational logic, reducing state errors
C) Makes code run on GPUs without CUDA | D) Replaces SQL syntax

Key: B — Encapsulation organizes complex pipeline state into modular, self-verifying entities.

Q5: What happens when executing 'record.is_valid = True' from outside the class?

A) Raises ImmutableError | B) Updates 'is_valid' on that specific instance
C) Modifies all instances globally | D) Crashes interpreter

Key: B — Standard Python class attributes are public by default and can be modified directly on the instance.

💡 Part 2: 5 Bonus Technical MCQs (Interview Focus)

BQ1: Which decorator creates a method that receives the Class itself as its first argument?

A) @staticmethod | B) @classmethod | C) @property | D) @abstractmethod

Key: B — @classmethod passes 'cls' as the first argument, commonly used for alternative constructors.

BQ2: What naming convention indicates a 'private' attribute in Python by name mangling?

A) _attribute | B) __attribute (double leading underscore) | C) attribute_ | D) __attribute__

Key: B — Double leading underscores trigger name mangling (_ClassName__attribute) to prevent collision.

BQ3: Which special dunder method defines developer-friendly string representations of objects?

A) __str__() | B) __repr__() | C) __dict__() | D) __format__()

Key: B — __repr__() provides an unambiguous string representation useful for debugging logs in pipelines.

BQ4: What is the main benefit of defining '__slots__' in a high-volume pipeline Record class?

A) Prevents subclassing | B) Restricts dynamic attributes and drastically reduces object memory footprint
C) Enables auto-multi-threading | D) Converts Python code into SQL

Key: B — __slots__ avoids per-instance __dict__ creation, saving significant RAM when instantiating millions of records.

BQ5: How does a '@staticmethod' behave inside a Python data pipeline class?

A) Receives 'self' automatically | B) Functions like a regular function scoped inside the class namespace